DEEPAVALI

PERSONALIZED INTERACTIVE DIWALI CELEBRATION PLATFORM

PRODUCT REQUIREMENTS DOCUMENT

Version 1.1 | Updated hosting and database architecture

Hosting: Vercel **Database:** Neon Serverless PostgreSQL **Storage:** Vercel Blob

Prepared as the master product and technical specification for implementation in AntiGravity IDE.

Core product promise: A visitor signs in to a creator-prepared account, receives a personalized Diwali rocket-and-fireworks greeting, and explores interactive experiences. The signature effect reconstructs the actual people in an uploaded photograph entirely from multicolored glowing particles.

Architecture decision: The application, server-side routes and deployment run on Vercel. Neon is provisioned through Vercel's Neon Marketplace integration and provides PostgreSQL. Vercel Blob stores private photographs and other media. No Supabase dependency is part of this revised architecture.

DOCUMENT CONTROL

Item	Details
Product name	DEEPAVALI - Personalized Interactive Diwali Celebration Platform
Document	Product Requirements Document (PRD)
Version	1.1
Status	Development specification
Primary developer	Hriday
IDE	AntiGravity IDE
Application hosting	Vercel
Database	Neon Serverless PostgreSQL, provisioned through Vercel Marketplace
Media storage	Vercel Blob, private access for student photographs
Primary framework	Next.js + React + TypeScript

VERSION 1.1 CHANGE SUMMARY

This revision replaces the earlier Supabase-based database and storage proposal. The application is hosted on Vercel, Neon provides the relational PostgreSQL database through Vercel's Marketplace integration, and Vercel Blob provides private media storage. Server-side application logic runs through Next.js Route Handlers / Server Actions on Vercel Functions.

Important product distinction: Vercel Postgres is no longer offered as a new standalone Vercel database product. New projects should install a PostgreSQL provider through the Vercel Marketplace; this PRD specifically selects the Neon integration.

CONTENTS

1. Product overview and objectives
2. Scope, users and assumptions
3. End-to-end user journey
4. Landing page and authentication
5. Personalized welcome cinematic
6. Celebration dashboard
7. Signature multicolor particle fireworks portrait
8. Additional Diwali experiences
9. Administrator dashboard
10. Vercel + Neon technical architecture
11. Database design and data model
12. Vercel Blob media storage and privacy
13. API and application behaviour
14. Security, privacy and reliability
15. UI, accessibility and responsive requirements

16. Performance and operational requirements
17. GitHub repositories and external references
18. Development roadmap
19. Testing and acceptance criteria
20. Deployment and release process
21. Risks, constraints and out-of-scope items
22. Final product definition

1. PRODUCT OVERVIEW AND OBJECTIVES

DEEPAVALI is a personalized, interactive Diwali celebration website. It replaces a static greeting page with a sequence of cinematic, account-specific experiences: a dark Diwali entrance, private credential entry, a rocket-powered personalized greeting, an interactive dashboard, and a collection of festive activities.

The signature feature is a photo-forming fireworks effect. After a firework explodes, its particles travel into sampled positions from an administrator-uploaded photograph. The particles retain sampled image colours and form recognizable people - faces, hair, clothing and body silhouettes - without displaying the source photograph as a normal image during this effect.

1.1 Vision

Create a magical digital Diwali celebration in which each invited visitor receives a personal welcome and can reveal shared memories through interactive light, fireworks and particle art.

1.2 Objectives

- Provide private access to creator-prepared student accounts. No public sign-up or student account creation.
- Show an introductory rocket and personalized HAPPY DIWALI greeting after successful authentication and before the dashboard.
- Build a recognizable, multicolored particle portrait from each account's assigned photographs.
- Support 3 to 5 photographs per student and cycle through them on repeated firecracker interactions.
- Provide additional activities such as diya lighting, lantern release, gift opening, memory gallery, secret message and a final celebration.
- Allow the administrator to create accounts, upload and order photographs, and configure personal greetings.
- Host the application, server-side logic, PostgreSQL integration and media storage within the Vercel ecosystem.

1.3 Success measures

- A valid student can authenticate and sees only their own assigned content.
- The welcome cinematic displays the administrator-configured display name, not the roll number unless explicitly configured that way.
- The particle engine forms recognizable people from particles and never flashes the ordinary source photograph during the effect.
- The user can trigger successive photographs without overlapping or corrupting the animation.
- The experience works on modern desktop and mobile browsers with adaptive visual quality.
- Unauthorized users cannot retrieve private photographs or administrator functions.

2. SCOPE, USERS AND ASSUMPTIONS

2.1 User roles

Role	Capabilities
Visitor / student	Enter an assigned roll number and DOB-format password; view their own greeting, photographs and messages; use celebration features; log out.
Administrator / creator	Authenticate to a protected admin area; create and manage student accounts; upload, order and remove photographs; edit greetings and messages; preview an account.

2.2 Account model

- All student accounts are created in advance by the administrator.
- Student login ID is the assigned college roll number.
- The requested password format is date of birth in DDMMYYYY format.

- The display name is a separate administrator-entered field and is the name shown in greetings.
- No student signup, public registration or account discovery is provided.
- Because date of birth is predictable, the product must use rate limiting, generic error messages, secure password hashing and session protections. An optional additional PIN is recommended if the photos or messages are sensitive.

2.3 Scope priorities

Priority	Features
P0 - Release essential	Landing page; student authentication; personalized welcome; dashboard; particle portrait engine; 3-5 photo sequence; admin account and media management; secure Vercel + Neon + Blob architecture.
P1 - Important	Diya lighting; secret message; gift box; memory wall; grand finale; sound settings.
P2 - Enhancement	Sky lantern wishes; Rangoli drawing; bulk CSV account import; advanced admin analytics.

2.4 Assumptions

- The administrator has permission to use every uploaded photograph and message.
- The first release is intended for a limited invited audience, not an unrestricted public social network.
- Particle image processing should run primarily in the visitor's browser to avoid expensive or time-limited server rendering.
- Neon stores structured data and references to media; image binary files are stored in Vercel Blob, not as large byte arrays in PostgreSQL.
- The application uses Vercel deployment environments for development previews and production.

3. END-TO-END USER JOURNEY

3.1 Main flow

Open website -> Diwali landing page -> ENTER -> login form -> server-side credential verification -> successful session -> full-screen darkening -> rocket launch -> explosion -> HAPPY DIWALI -> configured display name -> continue button -> celebration dashboard -> interactive activities -> grand finale.

3.2 Invalid login flow

The visitor submits missing or incorrect details. The server returns a generic authentication failure. The page shows a friendly retry message without revealing whether the roll number exists. Rate limits apply to repeated attempts.

3.3 Content loading flow

1. After authentication, the server resolves the authenticated student's internal ID.
2. The server retrieves the display name, greeting, active settings and ordered photo metadata for that student only.
3. The browser receives only the data needed for the current student's experience.
4. Photo assets are fetched through an authenticated Vercel Function route that checks the session and student ownership before streaming the private Blob.
5. The browser preprocesses each permitted image into particle target buffers. It may cache derived particle buffers in memory for the current session.
6. The original image is not drawn as a visible layer during the fireworks portrait sequence.

3.4 Navigation principles

- Keep the experience within a consistent celebration shell; avoid unnecessary page reloads.
- Each activity has a clear way to return to the main dashboard.
- Preserve the authenticated session while the visitor navigates.
- Provide sound, motion and fullscreen controls where appropriate.

• Do not start resource-heavy animations in hidden sections.

4. LANDING PAGE AND AUTHENTICATION

4.1 Landing page

• Full-screen dark night-sky composition with deep black / purple background.

• Floating golden particles, glowing diyas and subtle festive motifs.

• Large HAPPY DIWALI heading and short supporting line.

• Prominent ENTER control with a clear hover / press state.

• Responsive composition for phone portrait, tablet and desktop.

4.2 Login interface

Control	Requirement
ID input	Label: ENTER YOUR ID. Hint: Your college roll number. Text input; trim whitespace; validate required value.
Password input	Label: ENTER YOUR PASSWORD. Hint: Date of birth (DDMMYYYY). Use password input type; validate expected format on server.
Submit	UNLOCK YOUR CELEBRATION. Disable while request is pending; show progress state.
Error state	Generic message: 'Those details do not match. Please try again.' Do not disclose account existence.
Back control	Return to landing page without creating a session.

4.3 Authentication behaviour

1. Validate required fields in the browser for usability.
2. POST credentials to a server-side Next.js Route Handler or Server Action.
3. Normalize roll number consistently; do not transform the password unexpectedly.
4. Look up the account by a unique normalized roll number.
5. Verify the submitted password using a password-hashing library such as Argon2id. Never store DOB as plaintext.
6. Check account active status and apply login rate limiting.
7. Create a secure, HTTP-only, SameSite session cookie with an appropriate lifetime.
8. Return only the minimum user profile and redirect / transition into the welcome cinematic.

4.4 Admin authentication

Administrator authentication is separate from student authentication. The admin route must require an administrator role and must never rely only on hiding admin controls in the browser.

5. PERSONALIZED WELCOME CINEMATIC

This mandatory sequence runs after successful student login and before the main dashboard. The configured display_name field is used for the greeting.

Stage	Expected behaviour	Approx. duration
1. Darken	Login UI fades out; full viewport becomes nearly black; subtle particles appear.	0-1 sec
2. Launch	A glowing rocket rises from bottom centre with a trail and restrained smoke particles.	1-3 sec
3. Explosion	Rocket bursts near the middle; flash, radial particles, shockwave and secondary sparkles appear.	3-5 sec

Stage	Expected behaviour	Approx. duration
4. Greeting	HAPPY DIWALI appears, followed by the account display name.	5-7 sec
5. Hold	Greeting remains readable over subtle fireworks.	7-10 sec
6. Continue	ENTER MY DIWALI WORLD appears; click transitions into dashboard.	After greeting

Requirements

- Use a timeline controller to sequence events and prevent animation overlap.
- Support reduced-motion preference and a replay option after the first visit.
- Audio starts only after a user interaction where browser autoplay restrictions apply.
- Do not display the student's roll number as their name unless the administrator configured it as the display name.
- Keep the greeting readable on small screens and ensure the CTA remains accessible.

6. CELEBRATION DASHBOARD

The dashboard is the central activity hub. It should look like a premium cinematic Diwali environment rather than a conventional admin dashboard.

6.1 Header

- DEEPAVALI title / emblem.
- Personalized greeting using display_name.
- Sound and effects controls.
- Fullscreen control where supported.
- Logout action.

6.2 Main content

Display a short welcome line and visually distinct entry points for Fireworks, Light a Diya, Sky Lanterns, Surprise Gift, Memories, Rangoli, Secret Message and Grand Finale.

6.3 Navigation

- Use animated cards, floating icons or scene-integrated controls.
- Keep touch targets comfortable on mobile.
- Return to dashboard without losing the authenticated session.
- Load heavy activity modules on demand where practical.

7. SIGNATURE FEATURE: MULTICOLOR PARTICLE FIREWORKS PORTRAIT

7.1 Product definition

The user presses FIRE. A rocket launches from the bottom and explodes near the middle of the viewport. The explosion particles disperse, then move into target positions sampled from the next assigned photograph. The particles reconstruct the actual people in the photograph. The resulting portrait is composed entirely of particles; no normal photograph layer may be shown during this effect.

7.2 Visual direction

- Preserve sampled original RGB colours rather than converting the whole portrait to gold.

- Use multicolour light particles with subtle white / golden highlight sparkles.
- Make face, hair, clothing and body contours recognizable.
- Use a dark background to improve particle contrast.
- Keep background particles sparse so they do not obscure people.

7.3 Photo inputs

- Each student account supports 3 to 5 assigned photographs; fewer are allowed if configured.
- Accepted formats: JPEG, PNG and WebP. Validate actual file content, not only filename extension.
- Set an administrator-configurable file-size and pixel-dimension limit.
- Preserve aspect ratio and fit the subject into a safe portrait region.
- Administrator sets display order and may add captions for the memory gallery.

7.4 Image preprocessing pipeline

1. Fetch the authenticated student's private photo through a server-authorized media route.
2. Decode the image in the browser and resize to a rendering resolution suitable for the device.
3. Use Canvas pixel sampling to extract coordinates, RGB values, alpha and brightness.
4. Optionally use MediaPipe Image Segmenter to separate people from the background.
5. Optionally use MediaPipe Face Landmarker to assist with face-region weighting; it is not a guarantee of identity recognition.
6. Create a weighted target point cloud with greater density around eyes, mouth, face edges, hairline, clothing edges and body silhouette.
7. Generate particle buffers containing current position, target position, colour, size, velocity, opacity, depth and animation phase.
8. Cache processed buffers in memory for the current authenticated session where practical.

7.5 Weighted particle distribution

Region	Sampling strategy
Eyes, nose and mouth	Highest detail density; preserve local contrast and geometry.
Face boundary	High density to preserve recognizable outline.
Hair and hairline	High density; preserve dark/light colour variation.
Clothing	Medium-high density, retaining original colour blocks and folds where visible.
Body silhouette	High density along important contours.
Background	Low density or omit where segmentation identifies non-person regions.

7.6 Rocket and explosion

- Rocket launches from bottom centre and travels toward the target explosion point.
- Use a glowing head, trail, smoke / ember particles and acceleration.
- Explosion includes a flash, radial burst, varied velocities, gravity, drag, secondary crackles and a shockwave.
- Fireworks particles should be handed into the shared particle renderer where practical; avoid two competing portrait renderers.
- Sound effects must be optional and synchronized to the timeline.

7.7 Reconstruction

1. At explosion, create an outward-moving burst from the rocket's endpoint.
2. After the initial dispersion, assign burst particles to image target coordinates.
3. Apply smooth attraction / spring forces toward target positions.

4. Use interpolation with controlled easing; avoid abrupt teleportation.
5. Preserve original sampled colour throughout the transition.
6. Reveal face and body structure gradually so the person becomes recognizable.
7. Do not fade in the source photograph underneath the particles.

7.8 Hybrid portrait animation

Layer	Behaviour	Constraint
Shimmer	Independent brightness, opacity and occasional sparkle changes.	Must not flicker excessively or erase details.
Organic flow	Small noise-driven movement around target coordinates.	Keep displacement small near facial landmarks.
Breathing / floating	Very subtle global scale, depth and slow wave movement.	Portrait remains stable and recognizable.

7.9 Dissolve and next photograph

- Release attraction forces and apply outward dispersion from current positions.
- Add small sparkles and gradually reduce opacity.
- Reuse particle buffers and renderer for the next photograph.
- Cycle in administrator-defined order; after the last image, return to the first.
- Rapid clicks must queue or gracefully transition; they must not create concurrent render loops or corrupt buffers.

7.10 Particle state machine

READY -> ROCKET_LAUNCH -> EXPLOSION -> PARTICLE_DISPERSION -> IMAGE_RECONSTRUCTION -> PORTRAIT_IDLE -> PARTICLE_DISSOLVE -> NEXT_IMAGE -> IMAGE_RECONSTRUCTION. A central AnimationController is the sole owner of the active portrait transition state.

7.11 Performance baseline

Initial profiling targets: approximately 10,000 particles on capable desktop devices and 3,000-5,000 on mobile. These are starting points, not guaranteed minimums. Adapt based on measured frame time, GPU capability, viewport and device memory. Prefer WebGL2 / Three.js BufferGeometry and GLSL shader updates. Use a Canvas fallback with reduced complexity if WebGL is unavailable.

7.12 Critical acceptance rule

A portrait is not accepted merely because dots form a rough rectangle or silhouette. Test real sample photos and confirm that people are recognizable, original colour regions remain legible, facial structure survives the idle animation, and the original photo is never shown as a conventional image during the fireworks portrait.

8. ADDITIONAL DIWALI EXPERIENCES

8.1 Light a Diya

Show one or more unlit diyas. On tap, reveal a flame, flicker animation, warm glow and small floating embers. Allow the visitor to light several diyas. Display a short message such as 'May happiness and positivity illuminate your life.'

8.2 Sky Lanterns

Provide a short text input for a personal wish and a RELEASE button. On release, a lantern illuminates, rises slowly and fades into the night sky. Wishes remain private to the current visitor and are not published to other users.

8.3 Surprise Gift Box

Show a closed gift box. On interaction, shake the box, emit light, open the lid, release confetti and reveal the account's administrator-configured surprise message. Provide replay.

8.4 Memory Wall

Display the student's assigned photographs as a gallery with animated Polaroid-style cards, captions, enlarge interaction and responsive layout. Unlike the particle feature, this gallery may show normal photographs because its purpose is to view memories.

8.5 Rangoli Creator

Provide a drawing canvas supporting touch and mouse, colour selection, brush size, optional patterns, undo, redo, clear and optional glow. Treat this as a later-priority feature.

8.6 Secret Message

Show a sealed envelope. On click, animate the envelope opening and reveal a message written by the administrator for that account.

8.7 Grand Finale

Darken the screen, launch coordinated fireworks, show the visitor's display name and final personalized greeting, then offer CELEBRATE AGAIN, RETURN HOME and REPLAY FIREWORKS controls.

9. ADMINISTRATOR DASHBOARD

9.1 Access

A separate protected administrator authentication flow is required. Every admin operation must be authorized server-side. Do not rely on hidden UI elements for security.

9.2 Account creation fields

Field	Requirement
Roll number	Unique student login ID; normalized and indexed.
Display name	Name used in personalized greeting; separate from roll number.
Password	Administrator enters DOB-format password; hash immediately; never display stored hash.
Greeting message	Optional account-specific welcome text.
Secret message	Optional account-specific surprise message.
Status	Active / inactive.
Photos	Upload 3-5 photos; set order and captions.

9.3 Admin capabilities

- Create, view, search, edit, deactivate and delete student accounts.
- Upload, preview, replace, reorder and delete photographs.
- Edit display name, greeting and secret message.
- Preview a student's celebration through a protected preview mode.
- See total / active / inactive account counts and photo counts.
- Optional later enhancement: CSV bulk import with validation and duplicate reporting.

9.4 Preview mode

Preview must use the same student experience but be authorized as an administrator preview. Do not create a reusable student session or expose admin credentials to the browser. Clearly distinguish preview mode from a real student login.

10. VERCEL + NEON TECHNICAL ARCHITECTURE

10.1 Hosting decision

The whole application is deployed to Vercel. Next.js pages, React components, server-side route handlers and server actions are deployed as part of the Vercel project. Neon is the selected PostgreSQL provider, provisioned and connected through the Vercel Marketplace Neon integration. Vercel Blob is used for media storage.

Terminology note: 'Vercel Postgres' is no longer available for new projects. The correct current approach is to install a Postgres provider through Vercel Marketplace. This PRD selects Neon. Depending on the Marketplace installation mode, billing may be managed through Vercel or an existing Neon account may be linked.

10.2 Selected stack

Layer	Selected technology	Responsibility
Application framework	Next.js App Router	Routing, rendering, server components and client components.
Language	TypeScript	Typed application and engine code.
UI	React + CSS / Tailwind CSS	Responsive screens and reusable components.
Animation timelines	GSAP	Greeting sequence, scene transitions and UI animation.
3D / particles	Three.js	Single shared particle renderer, BufferGeometry and shader materials.
GPU	WebGL2 / GLSL	Efficient particle movement and rendering.
Image processing	Canvas API	Pixel sampling, resizing and target generation.
Person analysis	MediaPipe Image Segmenter; optional Face Landmarker	Segmentation and face-region weighting.
Server-side logic	Next.js Route Handlers / Server Actions on Vercel Functions	Authentication, admin actions, protected media delivery.
Database	Neon Serverless PostgreSQL	Accounts, settings, messages and media metadata.
Database driver	@neondatabase/serverless	Serverless-compatible PostgreSQL connection.
ORM / migrations	Drizzle ORM + drizzle-kit (recommended)	Typed schema, migrations and queries.
Media storage	Vercel Blob - private store	Student photos and private media.
Deployment	Vercel	Preview and production deployments.
Version control	GitHub	Source history and deployment integration.

10.3 Architecture diagram

VISITOR BROWSER -> VERCEL NEXT.JS APPLICATION -> VERCEL FUNCTIONS / ROUTE HANDLERS -> NEON POSTGRESQL. For media, the browser requests an authenticated media route; the Vercel Function verifies the session and student ownership, reads the private Vercel Blob object and streams it to the browser. The browser performs Canvas / MediaPipe preprocessing and renders particles locally using Three.js / WebGL.

10.4 Why Neon PostgreSQL

- Relational records fit students, photographs, settings and messages.
- Unique constraints and foreign keys enforce account and photo relationships.
- PostgreSQL transactions support consistent admin operations.
- Neon is serverless and designed to integrate with Vercel deployments.
- Preview database branches can isolate preview deployments when configured.

10.5 Database access rules

- Only server-side code connects to Neon. Never connect directly from browser code.
- Use the connection variables injected by the Vercel Marketplace integration.
- Use pooled connection strings / Neon serverless driver as recommended for serverless request patterns.
- Keep database credentials out of source control and client bundles.
- Use migrations in version control; do not make undocumented manual production schema changes.
- Use separate development / preview and production database branches or projects as appropriate.

10.6 No Supabase dependency

The revised architecture removes Supabase from the selected stack. Do not add Supabase Auth, Supabase Storage or Supabase database dependencies unless the product owner explicitly changes this decision.

11. DATABASE DESIGN AND DATA MODEL

Use PostgreSQL in Neon. Store structured data and metadata in PostgreSQL; store binary photo objects in private Vercel Blob. The database stores the Blob pathname and content metadata, not the full image binary.

11.1 Tables

Table	Key fields	Purpose
administrators	id UUID PK; username UNIQUE; password_hash; is_active; created_at	Protected creator/admin accounts.
students	id UUID PK; roll_number UNIQUE; display_name; password_hash; greeting_message; secret_message; is_active; created_at; updated_at	Predefined student accounts and personalized content.
student_photos	id UUID PK; student_id FK; blob_pathname; content_type; size_bytes; width; height; caption; display_order; is_active; created_at	Ordered photo metadata; binary files live in Blob.
student_settings	id UUID PK; student_id UNIQUE FK; music_enabled; effects_enabled; reduced_motion; created_at; updated_at	Per-account presentation preferences.
admin_audit_log	id UUID PK; admin_id FK; action; target_student_id; metadata JSONB; created_at	Optional record of account and media changes.

11.2 Relationships

- One student has many student_photos.
- One student has one student_settings row (created with defaults if omitted).
- Administrators can create and manage many students.
- Deleting a student must trigger a controlled cleanup workflow for related Blob objects and database rows.

11.3 Data constraints and indexes

- Unique index on normalized students.roll_number.
- Foreign key from student_photos.student_id to students.id with deliberate deletion policy.
- Unique constraint on (student_id, display_order) if each position must be unique.
- Indexes for student_id, is_active and display_order where they support common queries.
- Use UUID identifiers; never expose sequential internal IDs unnecessarily.
- Use created_at / updated_at timestamps in UTC.

11.4 Password storage

The submitted DOB-format password is verified against a strong password hash created by Argon2id or an appropriate password hashing implementation. Never store raw DOB, return password hashes to the browser or place passwords in CSV exports. Admin password reset must set a new hash and may invalidate existing sessions.

11.5 Migration strategy

1. Define the schema using Drizzle ORM.
2. Generate SQL migrations with drizzle-kit.
3. Review migrations before applying.
4. Apply migrations to development / preview database first.
5. Run tests against the migrated schema.
6. Apply production migrations through a controlled release step.
7. Keep migrations in the repository.

11.6 Example logical schema

The following is a logical example; exact column types and naming can be refined during implementation.

```
students(id UUID PRIMARY KEY, roll_number TEXT UNIQUE NOT NULL, display_name TEXT NOT NULL, password_hash TEXT NOT NULL, greeting_message TEXT, secret_message TEXT, is_active BOOLEAN NOT NULL DEFAULT TRUE, created_at TIMESTAMPTZ NOT NULL, updated_at TIMESTAMPTZ NOT NULL)
```

```
student_photos(id UUID PRIMARY KEY, student_id UUID REFERENCES students(id), blob_pathname TEXT NOT NULL, content_type TEXT NOT NULL, size_bytes BIGINT NOT NULL, width INTEGER, height INTEGER, caption TEXT, display_order INTEGER NOT NULL, is_active BOOLEAN NOT NULL DEFAULT TRUE, created_at TIMESTAMPTZ NOT NULL)
```

12. VERCEL BLOB MEDIA STORAGE AND PRIVACY

12.1 Storage choice

Use Vercel Blob for uploaded student photographs and other media. Create a private Blob store because the photographs are account-specific and should not be retrievable by anyone who knows or guesses a URL.

12.2 Upload flow

1. Administrator authenticates to the protected admin area.
2. Administrator selects a student and uploads a photograph.
3. Browser sends the file to a protected upload route or a secure upload flow.
4. Server verifies admin permission, file type, size and student association.
5. Server uploads to Vercel Blob using private access.
6. Server stores the returned Blob pathname and metadata in Neon.
7. Admin UI refreshes the ordered photo list.

12.3 Private read flow

1. Student is authenticated.
2. Browser requests a photo by an opaque photo ID, not by an arbitrary Blob pathname.
3. Server loads the photo metadata and verifies it belongs to the authenticated student (or authorized admin preview).
4. Server fetches the private Blob using the Vercel Blob SDK.
5. Server streams the content with appropriate content type and private cache headers.
6. Only then may the browser preprocess the image for the particle effect.

12.4 Storage security rules

- Use a private Blob store for student photographs.
- Do not expose BLOB_READ_WRITE_TOKEN or other storage credentials in client code.
- Never trust a client-supplied Blob pathname; resolve it from authorized database metadata.
- Authenticate and authorize every private read and write in the route handler itself.

- Use appropriate Cache-Control headers; for sensitive photo delivery prefer private, no-store unless a reviewed caching design is implemented.
- Validate file content, extension, MIME type, size and dimensions.
- Remove Blob objects when a photo is deleted and implement cleanup for failed database/storage operations.
- Do not put private photo URLs into public HTML, public JSON, logs or analytics.

12.5 Why Blob rather than database byte storage

Photographs are object files, not relational records. Blob storage is designed for file delivery and storage. PostgreSQL should store only metadata, ownership, ordering and Blob path references. This keeps database rows smaller and makes media lifecycle management clearer.

13. API AND APPLICATION BEHAVIOUR

Use Next.js Route Handlers / Server Actions for authenticated server-side operations. The following endpoint names are illustrative; AntiGravity may refine names while preserving behaviour.

Operation	Method / route example	Authorization
Student login	POST /api/auth/student/login	Public endpoint with rate limiting
Logout	POST /api/auth/logout	Authenticated session
Current student profile	GET /api/student/me	Student session
Student photo list	GET /api/student/photos	Student session; own records only
Private photo stream	GET /api/media/student-photo/[photoId]	Student session + ownership check
Admin login	POST /api/auth/admin/login	Public endpoint with rate limiting
Create student	POST /api/admin/students	Admin role
Update student	PATCH /api/admin/students/[id]	Admin role
Deactivate student	POST /api/admin/students/[id]/deactivate	Admin role
Upload photo	POST /api/admin/students/[id]/photos	Admin role
Reorder photos	PATCH /api/admin/students/[id]/photos/order	Admin role
Delete photo	DELETE /api/admin/photos/[photoId]	Admin role
Preview student	GET /api/admin/preview/[studentId]	Admin role; preview context

13.1 Response principles

- Return only fields required by the current screen.
- Never return password hashes, secret tokens or admin-only metadata to student clients.
- Use consistent error codes and user-friendly messages.
- Validate all input server-side using a schema validation library such as Zod.
- Use pagination for admin account lists if account counts grow.
- Log security-relevant failures without logging passwords or sensitive photo contents.

14. SECURITY, PRIVACY AND RELIABILITY

14.1 Authentication and authorization

- Hash passwords using Argon2id or a suitable password-hashing algorithm.
- Use secure, HTTP-only, SameSite cookies and HTTPS.

- Apply rate limiting to student and administrator login routes.
- Use generic login errors to prevent account enumeration.
- Check account active status on each protected session / data access as appropriate.
- Separate student and administrator roles and authorization checks.
- Use CSRF protections where required by the chosen mutation pattern.
- Invalidate sessions on logout and sensitive account changes.

14.2 Secrets

- Keep DATABASE_URL and any Neon credentials server-side.
- Keep Vercel Blob credentials server-side.
- Use Vercel environment variables for Development, Preview and Production.
- Never commit .env files or credentials to GitHub.
- Do not prefix secrets with NEXT_PUBLIC_.

14.3 Data privacy

- Only display the authenticated student's own content.
- Admin preview must be explicitly authorized.
- Obtain permission for photographs before uploading.
- Do not publish account lists, DOBs or private media.
- Provide a controlled account and photo deletion workflow.
- Minimize collection of personal data.

14.4 Reliability

- Handle database and Blob failures gracefully.
- Do not mark an upload complete until both Blob storage and metadata persistence have succeeded, or a cleanup / retry process is in place.
- Use transactions for related database changes.
- Use idempotent admin actions where practical.
- Keep a recoverable backup / restore plan for the Neon database and important media.
- Use structured logs without secrets or sensitive content.

15. UI, ACCESSIBILITY AND RESPONSIVE REQUIREMENTS

15.1 Visual direction

Premium cinematic Diwali: deep black and purple atmosphere, gold highlights, warm orange accents, multicolored particles, elegant typography, glowing diyas and restrained festive motifs. Avoid generic dashboard styling and excessive rectangular controls.

15.2 Responsive requirements

- Support modern Android and iOS browsers, tablets, laptops and desktop monitors.
- Use touch-friendly controls and responsive typography.
- Support portrait orientation and avoid horizontal overflow.
- Keep important controls reachable on small screens.

- Limit device pixel ratio and particle count on weaker hardware.

15.3 Accessibility

- Use semantic controls, labels and keyboard navigation.
- Maintain sufficient contrast and readable text.
- Provide mute and volume controls.
- Respect prefers-reduced-motion and provide a reduced-effects mode.
- Avoid intense flashing and rapid strobe effects.
- Ensure animations do not block access to essential controls.

16. PERFORMANCE AND OPERATIONAL REQUIREMENTS

16.1 Rendering

- Use one shared Three.js renderer / WebGL context for the particle portrait experience.
- Use BufferGeometry and GPU shaders rather than thousands of DOM nodes.
- Reuse geometry, materials and typed arrays.
- Dispose of textures, geometries, materials and animation loops when no longer needed.
- Pause nonessential rendering when the page is hidden.
- Use a reduced-complexity Canvas fallback where feasible.

16.2 Image processing

- Perform expensive pixel sampling and segmentation in the browser where practical.
- Resize large source images before pixel analysis.
- Consider Web Workers for preprocessing that would otherwise block the UI thread.
- Do not run image segmentation repeatedly for the same photo in one session.
- Cache processed particle target data in memory; avoid storing sensitive image-derived data unnecessarily.

16.3 Vercel serverless constraints

Keep request handlers short and bounded. Do not use Vercel Functions as long-running particle renderers or as a place to perform heavy per-frame graphics work. The browser handles interactive rendering. Server functions handle authentication, database queries, admin operations and authorized Blob delivery. Configure function duration and region based on current Vercel plan and measured needs.

16.4 Initial performance targets

Metric	Initial target / approach
Desktop frame rate	Aim for 60 FPS on capable devices; adapt particle quality based on measured frame time.
Mobile frame rate	Aim for 30-60 FPS depending on device capability.
Particle count	Start around 10,000 desktop and 3,000-5,000 mobile; benchmark and tune.
Initial page load	Aim for under 3 seconds on a suitable connection; optimize assets and code splitting.
Animation stability	No duplicate loops, memory leaks or buffer corruption after repeated interactions.
Graceful degradation	Reduced particle quality or Canvas fallback when WebGL is unavailable.

17. GITHUB REPOSITORIES AND EXTERNAL REFERENCES

Repositories are research references, not a requirement to install six independent packages. AntiGravity must inspect source code, dependencies, compatibility, maintenance status and licenses before adapting any implementation. Prefer one unified particle engine and one fireworks controller.

Repository	Potential use	Integration guidance
collidingScopes/particular-drift	Image-to-particle conversion, flow fields, GPU rendering	Primary reference for image sampling / flow; repository identifies MIT license.
shifatmasud/IMG-2-Particles-Morph-Physics-System	Image morphing and particle physics	Study the morphing model and compatibility before adapting.
bhuvanesh-m-dev/particle-portrait	Portrait point-cloud / facial structure ideas	GPL-3.0 identified; study as reference unless GPL obligations are acceptable.
travisjmac/grand-fireworks-js	Rocket launch and fireworks display	Legacy / showcase status noted; inspect maintenance and compatibility.
collidingScopes/forcefield	Particle scattering, repulsion and return behaviour	Reference for scattering / attraction; repository identifies MIT license.
toddegray/fireworks	Three.js procedural fireworks physics	Reference for trajectories, burst patterns and particle behaviour.

Repository URLs

https://github.com/collidingScopes/particular-drift

https://github.com/shifatmasud/IMG-2-Particles-Morph-Physics-System

https://github.com/bhuvanesh-m-dev/particle-portrait

https://github.com/travisjmac/grand-fireworks-js

https://github.com/collidingScopes/forcefield

https://github.com/toddegray/fireworks

17.1 External technology documentation

Next.js App Router: https://nextjs.org/docs/app

Next.js authentication guide: https://nextjs.org/docs/app/guides/authentication

Three.js documentation: https://threejs.org/docs/

Three.js BufferGeometry: https://threejs.org/docs/pages/BufferGeometry.html

MediaPipe Image Segmenter: https://ai.google.dev/edge/mediapipe/solutions/vision/image_segmenter

MediaPipe Face Landmarker: https://ai.google.dev/edge/mediapipe/solutions/vision/face_landmarker

GSAP documentation: https://gsap.com/docs/v3/

Neon for Vercel Marketplace: https://vercel.com/marketplace/neon

Vercel Postgres documentation / migration note: https://vercel.com/docs/postgres

Vercel Blob documentation: https://vercel.com/docs/vercel-blob

Vercel Blob private storage: https://vercel.com/docs/vercel-blob/private-storage

Vercel Marketplace storage integrations: https://vercel.com/docs/marketplace-storage

Vercel deployment environments: https://vercel.com/docs/deployments/environments

Drizzle ORM: https://orm.drizzle.team/docs/overview

Neon serverless driver: https://neon.com/docs/serverless/serverless-driver

17.2 Repository integration rules

- Do not install all six projects as independent runtime dependencies.

- Extract only the algorithms / shaders that are compatible with the unified engine.
- Keep required license and attribution notices.
- Do not copy GPL-licensed code into a differently licensed distribution without reviewing legal obligations.
- Pin dependency versions and test the combined renderer in the actual Next.js project.
- Document every adapted external module and its source.

18. DEVELOPMENT ROADMAP

Phase	Work	Deliverable
1. Foundation	Next.js, TypeScript, styling, Git, Vercel project and Neon integration.	Running app connected to Neon.
2. Data and admin	Drizzle schema / migrations, admin auth, student CRUD, private Blob upload.	Admin can create accounts and manage photos.
3. Student access	Student login, secure sessions, profile and own-photo retrieval.	Working predefined account system.
4. Entry experience	Landing page, login UI, rocket greeting and dashboard transition.	Complete personalized entry flow.
5. Portrait prototype	Image sampling, particle generation, segmentation and face weighting.	Recognizable image-to-particle prototype.
6. Particle motion	Attraction, dispersion, shimmer, flow and breathing.	Living particle portrait.
7. Fireworks	Rocket, trail, explosion, shockwave and dissolve.	Integrated photo-forming fireworks.
8. Other experiences	Diya, gift, secret message, memory wall, lantern, Rangoli.	Complete activity set by priority.
9. Finale	Full-screen celebration and replay.	Complete user journey.
10. QA and launch	Security, performance, device testing, production deployment.	Production release.

18.1 Required milestone discipline

1. Do not build the entire website before proving portrait reconstruction.
2. First deliver a standalone image-to-particle prototype using test photos.
3. Verify recognizability before adding the fireworks sequence.
4. Add rocket and explosion only after the particle portrait works.
5. Integrate the engine into the authenticated student experience after it is stable.
6. At each milestone, run the app, inspect the real visual result and test on mobile.

19. TESTING AND ACCEPTANCE CRITERIA

19.1 Authentication tests

- Valid roll number and password succeeds.
- Invalid credentials show generic error.
- Inactive account is rejected.
- Repeated failures are rate limited.
- Student cannot access admin routes.
- Student A cannot retrieve Student B's photos.
- Logout invalidates the session.

19.2 Particle portrait tests

- One-person and multiple-person photos are tested.
- Portrait and landscape photos preserve aspect ratio.
- People are recognizable from particle placement.
- Original colour regions are retained.
- No conventional source photo appears during the particle animation.
- Shimmer, flow and floating do not destroy facial structure.
- Dissolve and next-photo reconstruction work.
- Rapid repeated clicks queue or transition safely.
- WebGL resources are cleaned up.

19.3 Admin and storage tests

- Admin can create and edit accounts.
- Duplicate roll numbers are rejected.
- Admin can upload, reorder, replace and delete photos.
- Private Blob objects cannot be fetched without authorization.
- Unauthorized users cannot call admin upload or delete routes.
- Database metadata and Blob objects remain consistent after failures.

19.4 Responsive and performance tests

- Test Android phone, iPhone, tablet, laptop and desktop.
- Test portrait and landscape orientations.
- Measure frame rate and memory during repeated effects.
- Test reduced-motion mode and sound controls.
- Test WebGL unavailable / fallback behaviour.
- Verify no horizontal overflow and all controls are usable.

19.5 Release acceptance

A release is accepted only when all P0 requirements work, private student data is protected, the particle portrait is visually recognizable, repeated interactions are stable, and the production Vercel deployment has passed the agreed device and security checks.

20. DEPLOYMENT AND RELEASE PROCESS

20.1 Vercel project setup

1. Create a GitHub repository and push the Next.js application.
2. Import the repository into Vercel.
3. Configure the project for the Next.js framework.
4. Install the Neon integration from the Vercel Marketplace and select the intended account / billing mode.
5. Provision the Neon PostgreSQL database and confirm the connection variables are added to the Vercel project.
6. Create a Vercel Blob store with Private access and connect it to the project.
7. Configure environment variables for Development, Preview and Production.
8. Run database migrations against the intended Neon branch.

9. Deploy a Preview build and test authentication, admin operations and private media delivery.
10. Promote the verified build to Production.

20.2 Database integration details

Use the Vercel Marketplace Neon integration rather than assuming a new standalone Vercel Postgres product exists. The integration provisions / links Neon and injects connection variables such as DATABASE_URL. Confirm the actual variable names in the Vercel project settings and follow the installed integration's current instructions. Use the Neon serverless driver and a serverless-compatible connection pattern.

20.3 Preview and production separation

- Use Vercel Local, Preview and Production environments.
- Prefer a separate Neon branch for preview deployments where supported and enabled.
- Never allow a preview deployment to write to production student data by accident.
- Keep production secrets separate from development secrets.
- Run migrations and smoke tests before promoting a release.

20.4 Production checklist

- Production environment variables configured.
- Neon database reachable from server-side routes.
- Database migrations applied.
- Private Blob store connected.
- Admin account secured.
- Student accounts imported / created.
- Photo ownership checks verified.
- Custom domain and HTTPS configured if used.
- Error logging and backup / restore process reviewed.
- Final mobile and desktop smoke tests passed.

21. RISKS, CONSTRAINTS AND OUT-OF-SCOPE ITEMS

21.1 Key risks

Risk	Mitigation
Faces not recognizable as particles	Use segmentation, weighted sampling, edge/detail preservation and iterative visual testing. Do not rely only on uniform pixel sampling.
GPU performance on phones	Adaptive particle counts, limited pixel ratio, shader updates and reduced-motion / fallback mode.
Private photos exposed	Private Blob store, authenticated server route, ownership checks and no public path disclosure.
DOB is weak credential	Hash it, rate-limit attempts, use generic errors and consider an additional private PIN.
Repository incompatibility or license conflict	Inspect code, dependencies and license before adapting; maintain notices; avoid copying GPL code without review.
Serverless request limits	Keep graphics in the browser; keep server functions short and focused on auth, database and media access.
Blob and database inconsistency	Use compensating cleanup / retry logic for partial upload or deletion failures.

21.2 Constraints

- The project is intended to remain within the Vercel ecosystem for hosting, database integration and media storage.

- Neon PostgreSQL is an external managed database provider integrated through Vercel Marketplace; it is not a database engine physically running inside the Vercel application runtime.
- Vercel Functions are not intended to render long-running per-frame animations.
- Browser GPU support and performance vary by device.
- Facial landmark models assist geometry and sampling but cannot guarantee perfect reconstruction for every photo.

21.3 Out of scope for initial release

- Public user registration.
- Public social feed or public sharing of private photos.
- AI-generated replacement faces or image generation.
- Real-world fireworks control or hardware integration.
- Complex analytics unrelated to the celebration.
- Persistent storage of visitors' free-text lantern wishes unless separately approved.

22. FINAL PRODUCT DEFINITION

DEEPAVALI is a personalized interactive Diwali celebration platform hosted on Vercel. It uses Next.js and TypeScript for the application, Neon Serverless PostgreSQL for structured account and content data, Vercel Functions for server-side operations, and Vercel Blob private storage for photographs.

Visitors authenticate using administrator-created accounts. After login, a cinematic rocket launches and explodes, revealing HAPPY DIWALI and the administrator-configured display name. The visitor then enters a festive dashboard containing interactive Diwali experiences.

The signature feature transforms assigned photographs into recognizable human portraits made entirely from multicolored glowing particles. The portrait shimmers, flows and floats subtly, then dissolves so the next photograph can form. The original photograph is never displayed as a normal image during this effect.

The project succeeds when the complete journey is cohesive, personal, visually memorable, secure, responsive and reliable - with particle portrait quality taking priority over decorative extras.

END OF DOCUMENT - DEEPAVALI PRD VERSION 1.1

Updated architecture: Vercel Hosting + Neon Serverless PostgreSQL + Vercel Blob Private Storage + Next.js / TypeScript.